THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

BSE4610-

Building Informatics

Dr. Zhiling GUO (ZLG)

Assistant Professor

Department of Building Environment and Energy Engineering (BEEE)

Semester 1, 2026/27

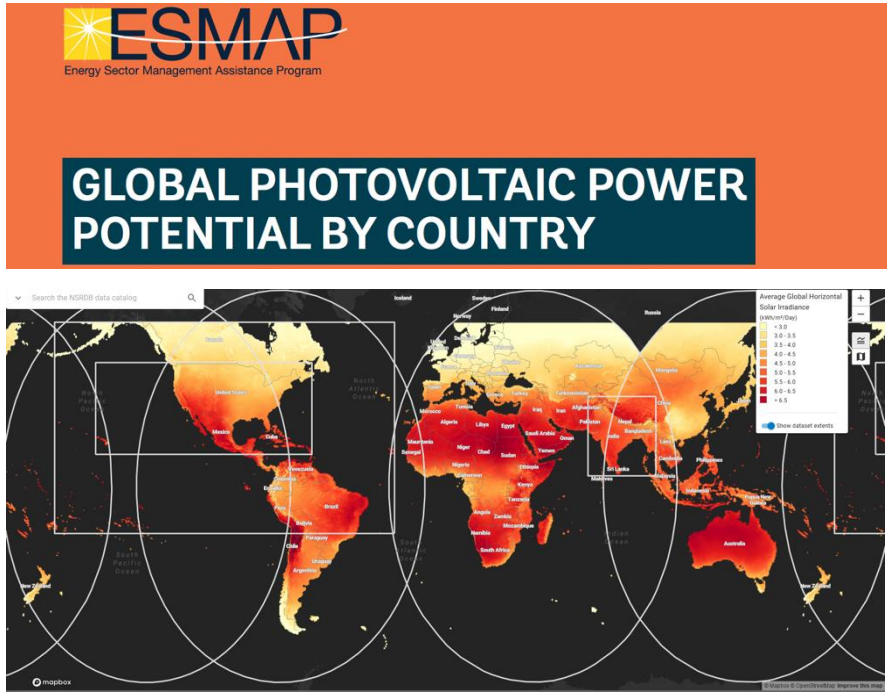


BSE4610 Building Informatics

Week 7: Mini Project

Today's focus:

- Python basics
- Mini Hands-on
- Mini Project





Prof. Zhiling Guo (ZLG)

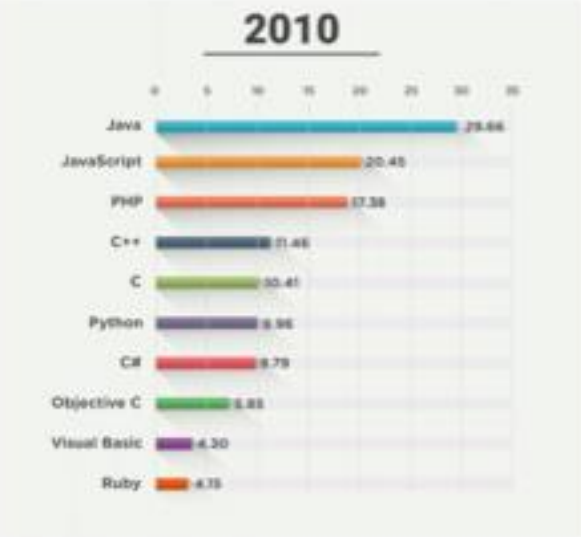
Department of Building Environment and Energy Engineering

Research interests:

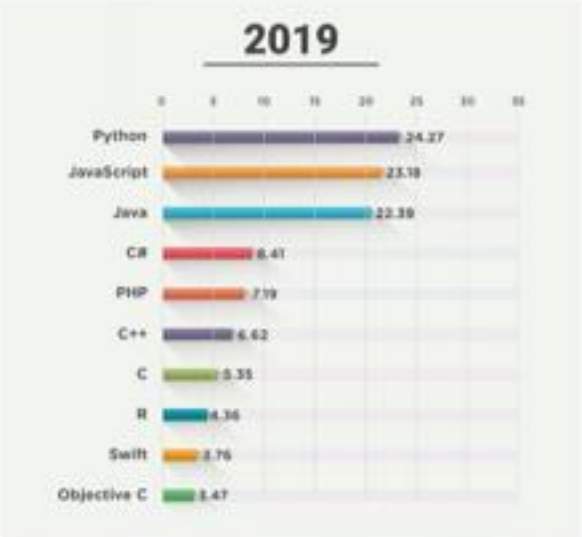
- Machine Learning,
- Geoscience,
- AIoT,
- Renewable Energy Technologies.

Part 1 - Python Basics

The Evolution of Programming Languages in Past 10 Years







2026

Worldwide, Oct 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	53.9 %	+24.9 %
2		Java	12.98 %	-0.1 %
3		C/C++	7.97 %	-5.0 %
4		R	3.87 %	-2.4 %
5	↑↑↑↑↑↑	Objective-C	3.51 %	+1.6 %
6	↓	JavaScript	2.81 %	-3.2 %
7	↑	PHP	1.74 %	-1.6 %
8	↑	Rust	1.56 %	-1.3 %
9	↑	Ada	1.55 %	-1.1 %
10	↓↓↓	Swift	1.47 %	-2.3 %
11	↓↓↓↓↓	C#	1.46 %	-2.4 %
12	↓	TypeScript	1.33 %	-0.7 %
13		Matlab	0.82 %	-0.8 %
14	↑↑	Ruby	0.7 %	-0.4 %

Introduction to Python

Python is a popular **programming language**.

- Developed by Guido van Rossum in 1989, and formally released in 1991
- An **open source**, **object oriented** programming language
- **Powerful libraries** (numpy, scikit-learn, pytorch, matplotlib, and so on)
- Powerful interfaces to integrate other programming languages (C/C++, Java, and many other languages)
- Programming language of the year 2010



```
print("Hello World")
```

Other programming language are C, C++, Java and so on.

Print "Hello World" via different programming languages, the code of Python is very simple:

```c
#include <stdio.h>

int main() {
   printf("Hello World");
   return 0;
}
```

```cpp
#include <iostream>

int main() {
    std::cout << "Hello World";
    return 0;
}
```

```java
import java.io.*;

class GFG {
    public static void main (String[] args) {
        System.out.println("Hello World");
    }
}
```

Background for programming language

What can a computer actually understand?

- The computers used nowadays can understand only binary number (i.e. 0 and 1)
- Computers use voltage levels to represent 0 and 1
- Non-return-to-zero level (NRZL) and Non-return-to-zero inverted(NRZI) coding
- The instructions expressed in binary code is called **machine language**

0 0 0 1 numerical value 2^0

0 0 1 0 numerical value 2^1

0 1 0 0 numerical value 2^2

1 0 0 0 numerical value 2^3

Low level language – Assembly Language

An **assembly language** is a low-level programming language, in which there is a very strong (generally one-to-one) correspondence between the language and machine code instructions.
Assembly language is converted into executable machine code by a utility program referred to as an **assembler**

```
                        ****************************************
                        * FUNCTION: INHEX - INPUT HEX DIGIT
                        * INPUT: none
                        * OUTPUT: Digit in acc A
                        * CALLS: INCH
                        * DESTROYS: acc A
                        * Returns to monitor if not HEX input

C01E 8D F0      INHEX   BSR     INCH        GET A CHAR
C020 81 30              CMP A   #'0         ZERO
C022 2B 11              BMI     HEXERR      NOT HEX
C024 81 39              CMP A   #'9         NINE
C026 2F 0A              BLE     HEXRTS      GOOD HEX
C028 81 41              CMP A   #'A
C02A 2B 09              BMI     HEXERR      NOT HEX
C02C 81 46              CMP A   #'F
C02E 2E 05              BGT     HEXERR
C030 80 07              SUB A   #7          FIX A-F
C032 84 0F      HEXRTS  AND A   #$0F        CONVERT ASCII TO DIGIT
C034 39                 RTS

C035 7E C0 AF   HEXERR  JMP     CTRL        RETURN TO CONTROL LOOP
```

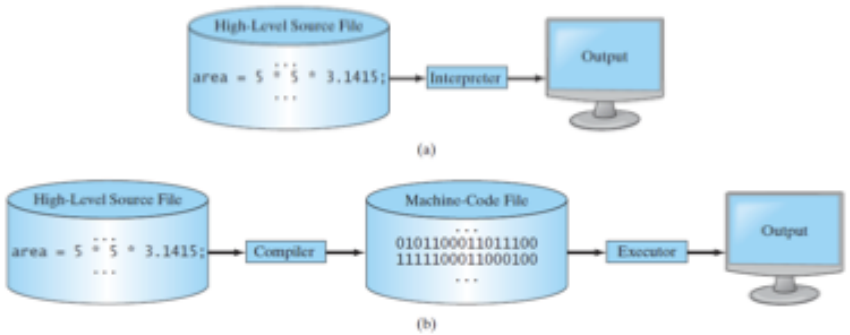
Language efficiency vs. development efficiency

- High level languages cannot be executed directly
- High level languages must be converted into low level languages first
- Lower level languages have higher language efficiency (they are faster to run on a computer)
- Higher level languages have higher development efficiency (it is easier to write programs in these languages)

Interpreter vs. compiler

- **Interpreter** is a computer program that directly executes, i.e. performs, instructions written in a programming or scripting language, without previously compiling them into a machine language program

- **Compiler** is a computer program (or a set of programs) that transforms source code written in a programming language (the source language) into another computer language (the target language), with the latter often having a binary form known as object code



Install Python

The formal website of Python, you can download Python on it:
https://www.python.org/



- Step 1: Select Version to Install Python;
- Step 2: Downloading the Python Installer;
- Step 3: Running the Executable Installer;
- Step 4: Verify the Python Installation in your computer.

```
(base) ■  ■  ■■  ■  ■-iMac ~ % python --version
Python 3.8.8
(base) ■  ■  ■■  ■  ■-iMac ~ % python
Python 3.8.8 (default, Apr 13 2021, 12:59:45)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>> exit()
(base) ■  ■  ■■  ■  ■-iMac ~ % vim test_python.py
(base) ■  ■  ■■  ■  ■-iMac ~ % python test_python.py
Hello from Python file
(base) ■  ■  ■■  ■  ■ iMac ~ %
```

check Python version

enter Python command line

Python code

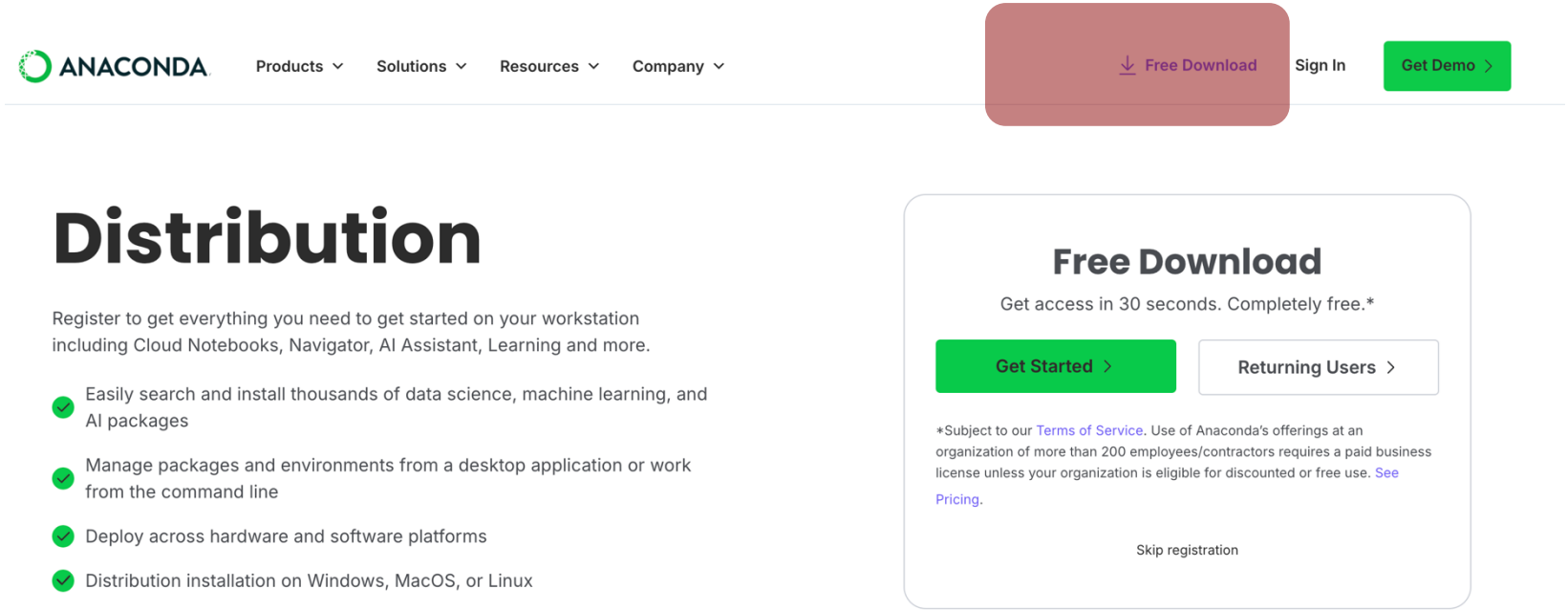
exit Python command line

run a Python file

```
test_python.py
1 print("Hello from Python file")
2
```



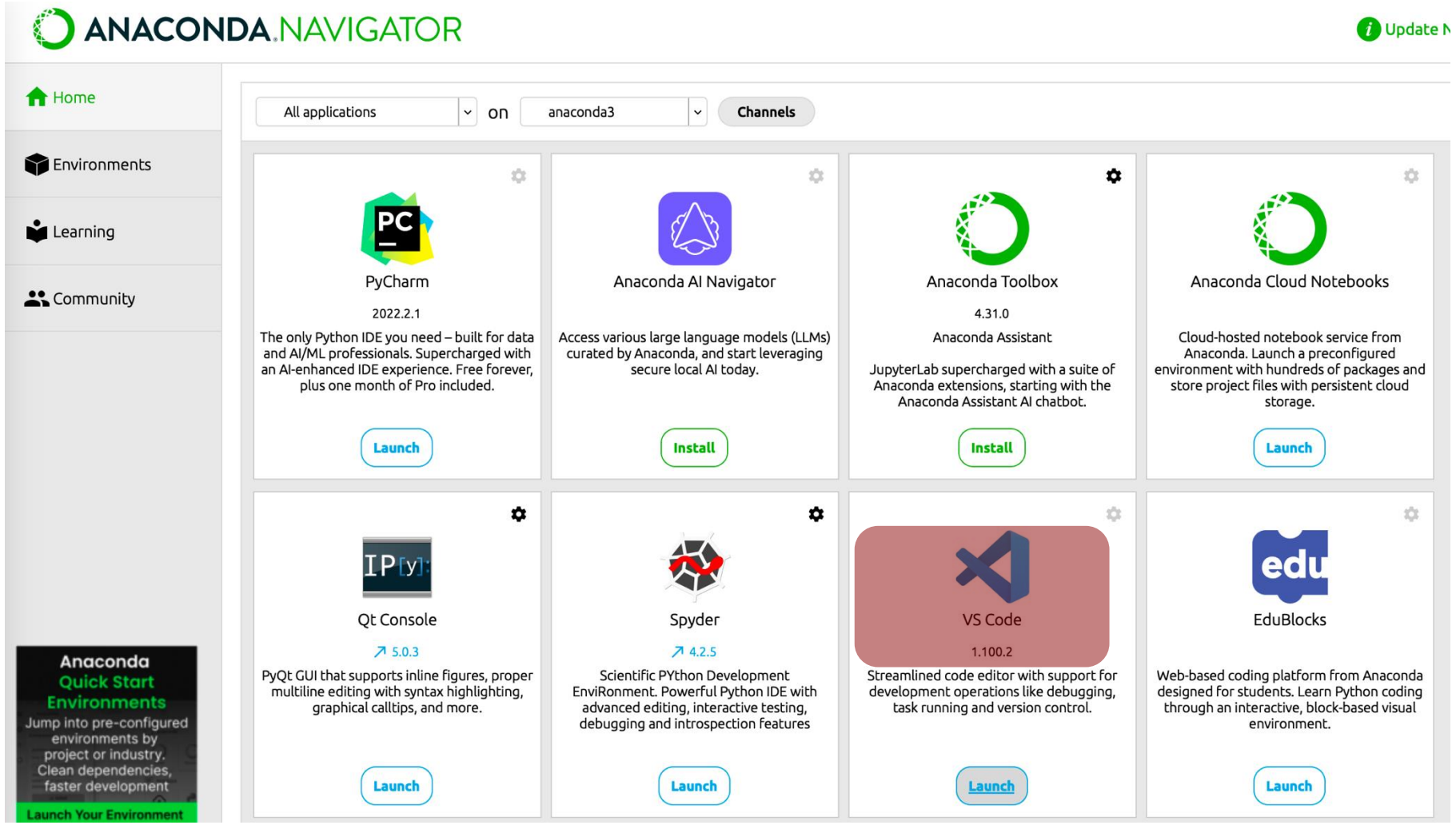
Install Python: Anaconda for a Trial



SCAN ME

Anaconda *is an open-source distribution designed for data science, machine learning, and scientific computing, which simplifies package management and deployment for the Python and R programming languages.*

Launch Python: Spyder for a Trial



Basics of Python

Python **syntax** can be executed by writing directly in the Command Line:

```
>>> print("Hello, World!")
Hello, World!
```

Or by creating a python file on the server, using the .py file extension, and running it in the Command Line:

```
C:\Users\Your Name>python myfile.py
```

Indentation refers to the spaces at the beginning of a code line. Where in other programming languages the indentation in code is for readability only, the **indentation in Python is very important**. Python uses indentation to indicate a block of code.

```
if 5 > 2:
  print("Five is greater than two!")
```



```
if 5 > 2:
print("Five is greater than two!")
```


Python has commenting capability for the purpose of in-code documentation. **Comments** start with a hashtag #, comment block with quotation mark ", and Python will render the rest of the line as a comment.

```
# This is a comment.
print("Hello from Python file")

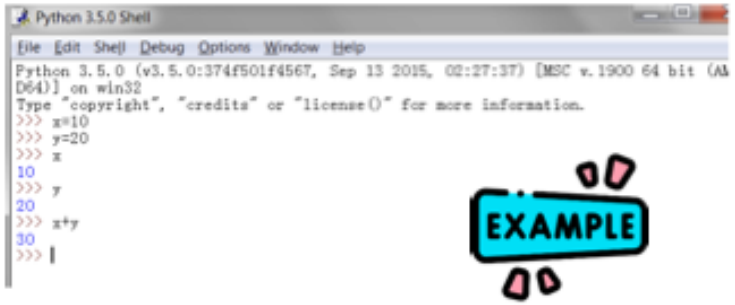
"""
This is also a comment,
but a block.
"""
print("Hello again")
```



Variables

A variable is a **named space** in the memory where a programmer can store data and later retrieve the data using the variable name

- Variable names are determined by programmers
- The **value** of a variable can be changed later in a program



Constants

- Fixed values such as numbers and letters are called **constants**, since their values won't change
- String constants use single-quotes (') or double-quotes (")

Assignment statement

- Assign a value to a variable using the **assignment operator** (=)
- An assignment statement consists of an expression on the **right hand side**, and a variable to store the result

Rules for defining variables in Python

- Must **start with a letter or underscore _**
- Can **only contain letters, numbers and underscore**
- **Case sensitive**
- Good: apple, car, myNumber123, _light
- Bad: 456aaa, #ab, var.12
- Different: apple, Apple, APPLE

Data type

Data Type	Description	Mutability
int / float / complex	Numeric values	Immutable
str	Text strings	Immutable
bool	Truth values (True/False)	Immutable
list	Ordered collection of elements	Mutable
tuple	Ordered collection of elements	Immutable
dict	Key-value pairs	Mutable
set	Unique collection of elements	Mutable

Python knows what type everything is
- Some operations are prohibited on certain types
- You cannot "add 1" to a string
- We can check the type of something using function **type()**

```
>>> x = 7 * 3 * 2
>>> y = "is the answer to the ultimate question of life"
>>> print(x, y)          # Check what x and y are.
42 is the answer to the ultimate question of life
>>> x, y                 # Quicker way to check x and y.
(42, 'is the answer to the ultimate question of life')
>>> type(x), type(y)     # Check types of x and y.
(<class 'int'>, <class 'str'>)
>>> # Set x and y to new values.
>>> x = x + 3.14159
>>> y = 123212132131231231231 2 * 9873423789237438297
>>> print(x, y)          # Check what x and y are.
45.14159 1216525596507164987120868363073549341266 4
>>> type(x), type(y)     # Check types of x and y.
(<class 'float'>, <class 'int'>)
```



Numbers in Python generally have two types.
- **Integers**: 1, 2, 100, -20394209
- **Floating point numbers**: 2.5, 3.7, 11.32309, -30.999

Boolean type

Python contains a built-in Boolean type, which takes two values True/False
- Number 0 can also be used to represent False.
- All other numbers represent True

Characteristics of data type in Python

- The type of a variable can be dynamically changed
- A variable's type is determined by the value that is last assigned to the variable

Reserved words

Python has a set of keywords that are **reserved words** that cannot be used as variable names, function names, or any other identifiers:

Keywords in Python programming language

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

```
Python 3.8.8 (default, Apr 13 2021, 12:59:45)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> is = 1
  File "<stdin>", line 1
    is = 1
       ^
SyntaxError: invalid syntax
>>> a = 1
>>>
```



Operators

Operators are used to perform operations on variables and values.

Python divides the operators in the following groups:

1. Arithmetic operators
2. Assignment operators
3. Comparison operators
4. Logical operators
5. Identity operators
6. Membership operators
7. Bitwise operators

1. **Arithmetic operators** are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

2. **Assignment operators** are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3
:=	print(x := 3)	x = 3 print(x)

Operators – Cont.

3. **Comparison operators** are used to compare two value:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

4. **Logical operators** are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

5. **Identity operators** are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Operators – Cont.

6. **Membership operators** are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

7. **Bitwise operators** are used to compare (binary) numbers:

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	x & y
|	OR	Sets each bit to 1 if one of two bits is 1	x | y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

Operator Precedence

Operator precedence describes the order in which operations are performed. The precedence order is described in the table below, starting with the highest precedence at the top:

Operator	Description
`()`	Parentheses
`**`	Exponentiation
`+x  -x  ~x`	Unary plus, unary minus, and bitwise NOT
`*  /  //  %`	Multiplication, division, floor division, and modulus
`+  -`	Addition and subtraction
`<<  >>`	Bitwise left and right shifts
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`==  !=  >  >=  <  <=  is  is not  in  not in`	Comparisons, identity, and membership operators
`not`	Logical NOT
`and`	AND
`or`	OR

EXAMPLE

```
a = 8 ** 3 / (2 * (1 + 3)) ** 2
print(a)
```

Data type to store collections of data

Lists are used to store multiple items in a single variable. Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage. Lists are created using square brackets:

Create a List:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

A **tuple** is a collection which is ordered and **unchangeable**. Tuples are written with round brackets:

Create a Tuple:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

A **set** is a collection which is **unordered, unchangeable** (set items are unchangeable, but you can remove items and add new items), and **unindexed**. Sets are written with curly brackets:

Create a Set:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Dictionaries are used to store data values in key: value pairs. A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

Create and print a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Flow

Sequential flow

when statements (i.e. line of code) execu te one after another without conditions o r loops then this is called sequential exec ution.

```
15  # Sequential flow
16  a = 1
17  b = 2
18  print(a+b)
```

Repeated flow: for

A **for loop** is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
31  # Repeated flow: for
32  for x in range(6):
33    print(x)
```

Conditional flow: if elif else

Python supports the usual logical conditions, these conditions can be used in several ways, most commonly in "if statements"

```
20  # Conditional flow: if elif else
21  a = 200
22  b = 33
23  if b > a:
24    print("b is greater than a")
25  elif a == b:
26    print("a and b are equal")
27  else:
28    print("a is greater than b")
```

Repeated flow: while

With the **while loop** we can execute a set of statements as long as a condition is true.

```
36  # Repeated flow: while
37  i = 1
38  while i < 6:
39    print(i)
40    i += 1
```

Function

A **function** is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

In Python a function is defined using the **def** keyword:

```
1 usage
def my_function():
    print("Hello from a function")


my_function()
```

Define a function

Call this function

Running results:

```
/usr/bin/python3 /Users
Hello from a function
```

Information can be passed into functions as **arguments**. Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma. The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

A function with argument:

```
3 usages
def my_function(fname):
    print("Hello, " + fname)


my_function("World")
my_function("Hong Kong")
my_function("PolyU")
```

Running results:

```
/usr/bin/python3 /U
Hello, World
Hello, Hong Kong
Hello, PolyU
```

OO - Class

Python is an **object oriented (OO)** programming language. Almost everything in Python is an object, with its properties and methods. A Class is like an object constructor, or a "blueprint" for creating objects.

All **classes** have a function called __init__(), which is always executed when the class is being initiated. Use the __init__() function to assign values to object properties, or other operations that are necessary to do when the object is being created.

To create a class, use the keyword **class**:

```
12  class Person:
13      def __init__(self, name, age):
14          self.name = name
15          self.age = age
16
17
18  p1 = Person("Paul", 34)
19
20  print(p1.name)
21  print(p1.age)
```

Create a class

Create an object from the class

Running results:

```
/usr/bin/python3
9
```



An example of Python code

```python
class myData:
    def __init__(self, arr):
        self.arr = arr

    def bubbleSort(self):
        n = len(self.arr)

        # For loop to traverse through all
        # element in an array
        for i in range(n):
            for j in range(0, n - i - 1):

                # Range of the array is from 0 to n-i-1
                # Swap the elements if the element found
                #is greater than the adjacent element
                if self.arr[j] > self.arr[j + 1]:
                    self.arr[j], self.arr[j + 1] = self.arr[j + 1], self.arr[j]

# Example to test the above code
arr = [ 2, 1, 10, 23 ]
# Create an object
data_object = myData(arr)
# Call the function to order an array "arr"
data_object.bubbleSort()

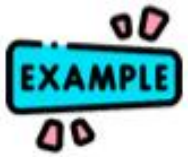
print("Sorted array is:")
for i in range(len(data_object.arr)):
    print("%d" % data_object.arr[i])
```

A class

for loop

if statement

create an object



Running results:

```
/Applications/anaconda3/bin/python
Sorted array is:
1
2
10
23
```

Import

If you quit from the Python interpreter and enter it again, the definitions you have made (functions and variables) are lost. Therefore, if you want to write a somewhat longer program, you are better off using a text editor to prepare the input for the interpreter and running it with that file as input instead. This is known as creating a ***script***.

A module is a file containing Python definitions and statements. The **file name is the module name** with the suffix .py appended. And use module name to import the module.

```
# Fibonacci numbers module

def fib(n):    # write Fibonacci series up to n
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):   # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result
```

```
>>> import fibo
```

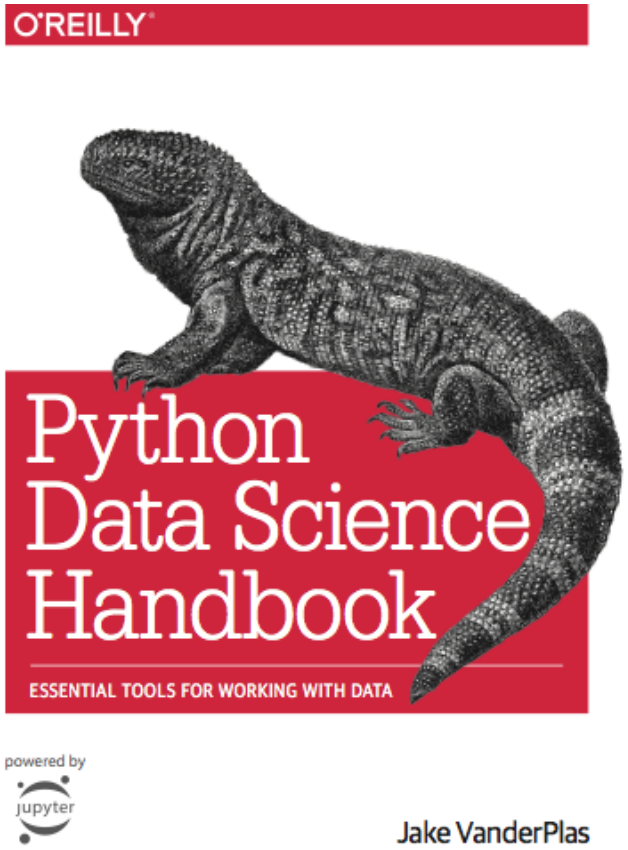
```
>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

https://docs.python.org/3/tutorial/modules.html#

Python Data Science Handbook



SCAN ME

https://jakevdp.github.io/PythonDataScienceHandbook/

Part 2 -
Mini Hands-on: Notebook Skeleton and Tasks

Mini Hands-on: Notebook Skeleton and Tasks

Notebook Sections

1. **Setup & imports**: seed, paths, libraries
2. **Load data**: parse timestamps, merge energy + weather
3. **Cleaning**: missing data report, validation checks
4. **Feature engineering**: calendar, weather, lags
5. **Time-based split**: baseline models (persistence, seasonal)
6. **Models**: OLS/Ridge + Random Forest; metrics
7. **Error analysis**: by hour/season; visualization
8. **Interpretability**: PDP/SHAP for key features
9. **Save artifacts**: README checklist, documentation

Cell [1]: Setup

```
# Import libraries and set seed
import pandas as pd
import numpy as np
from sklearn.model_selection import TimeSeriesSplit import
matplotlib.pyplot as plt

RANDOM_SEED =
np.random.seed(RANDOM_SEED)
```

In-class Tasks (10-15 min)

Complete these tasks using the provided starter CSV and partially filled notebook.

1 Implement seasonal naïve baseline

```
# Seasonal naïve (same hour, previous day) def
seasonal_naive(y, lag=24):
    y_pred = y.shift(lag)
    return y_pred
```

2 Create lag t-24 and a 24h rolling mean

```
# Create lag features
df['load_lag24'] = df['load'].shift(24)
df['load_rolling24'] = df['load'].rolling(24,
closed='left').mean()
```

3 Compute MAE/RMSE and compare to persistence

```
from sklearn.metrics import mean_absolute_error,
mean_squared_error
mae_persist = mean_absolute_error(y_test, y_persist)
rmse_persist = np.sqrt(mean_squared_error(y_test,
y_persist))
mae_seasonal = mean_absolute_error(y_test, y_seasonal)
rmse_seasonal = np.sqrt(mean_squared_error(y_test,
```

Setup and Imports

1 Building Energy Prediction: A Hands-On Guide

1.1 1. Setup and Imports

First, we'll import the necessary libraries and set a random seed for reproducibility.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, TimeSeriesSplit
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.preprocessing import StandardScaler
import shap

# Set plot style
sns.set_style("whitegrid")

# Seed for reproducibility
np.random.seed(42)
```

Load Data

1.2 2. Load Data

Next, we load our synthetic dataset. We'll parse the timestamps and set them as the index.

[2]:
```
df = pd.read_csv('building_data.csv', parse_dates=['timestamp'],␣
↪index_col='timestamp')
print("Data loaded successfully!")
df.head()
```

```
Data loaded successfully!
```

[2]:

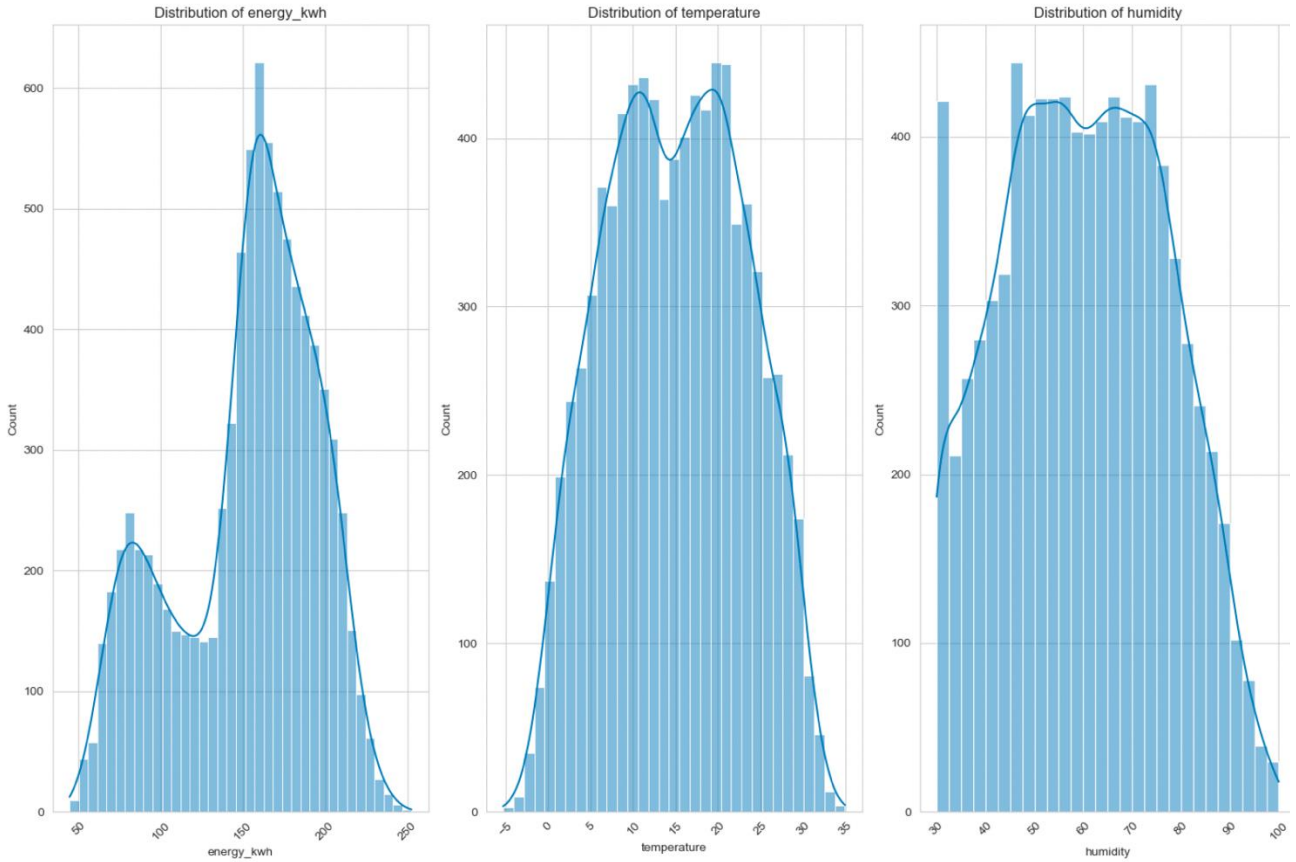
timestamp	energy_kwh	temperature	humidity
2023-01-01 00:00:00	96.775844	2.213827	90.429407

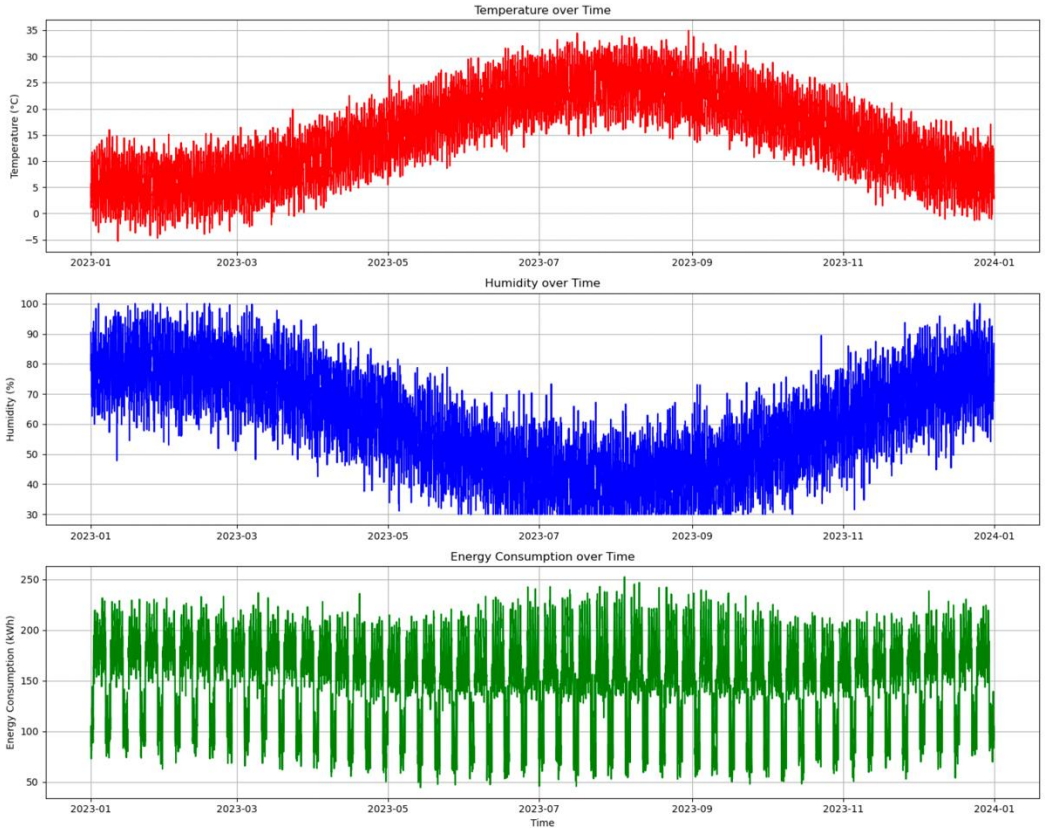
Data Visualization

1.3 3. Visualize Data

Next, we visualize data.

[3]:
```
# Visualize all columns in the dataframe
```





Cleaning Rules and Outlier Handling

Cleaning Actions

- Remove duplicates; sort index
- Negative kWh: flag → if rollover suspected, correct; else set NaN
- Short gaps (<2h): forward-fill; long gaps: leave NaN and flag

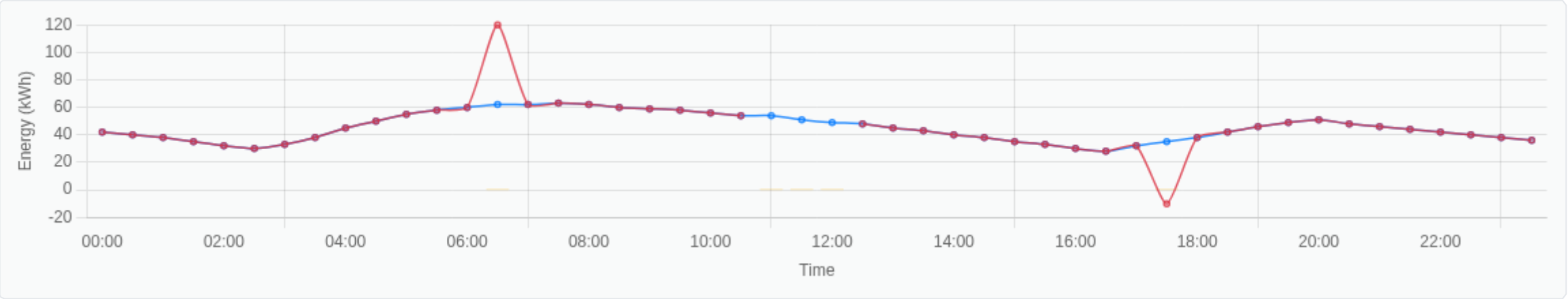
Remove Duplicates | Flag Negative Values | Forward-Fill Short Gaps | NaN Long Gaps

Auditing

- Add columns: `is_imputed`, `is_outlier`, `gap_length`
- Track all changes and their justification
- Document cleaning parameters for reproducibility

```
df['is_imputed'] = False df.loc[imputed_idx, 'is_imputed'] =
True
df['gap_length'] = df['kWh'].isna().astype(int).groupby(
        (df['kWh'].notna().astype(int).diff() != 0).cumsum()
).transform('sum')
```

Before & After Cleaning



Original Data (with spikes/gaps) Cleaned Data Flagged Regions

Data Preprocessing & Feature Engineering

1.4 4. Data Cleaning

Handle missing values, outliers, and inconsistencies in the data.

[7]:
```
print("Missing values before cleaning:")
print(df.isnull().sum())

# Fill missing values using forward-fill
```



```
df.ffill(inplace=True)

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values before cleaning:
energy_kwh     88
temperature    88
humidity       88
dtype: int64
Missing values after cleaning:
energy_kwh     0
temperature    0
humidity       0
dtype: int64
```

1.5 5. Feature Engineering

We'll create calendar, weather, and lag features to help our models capture patterns in the data.

:
```
def create_features(df):
    df['hour'] = df.index.hour
    df['day_of_week'] = df.index.dayofweek
    df['month'] = df.index.month
    df['quarter'] = df.index.quarter
    df['day_of_year'] = df.index.dayofyear
    df['is_weekend'] = (df.index.dayofweek >= 5).astype(int)

    # Lag features for energy
    df['energy_lag_24h'] = df['energy_kwh'].shift(24)
    df['energy_lag_1w'] = df['energy_kwh'].shift(24 * 7)

    return df

df = create_features(df)
df.dropna(inplace=True) # Drop rows with NaNs from lag features
df.head()
```

:

timestamp	energy_kwh	temperature	humidity	hour	day_of_week	\
2023-01-08 00:00:00	78.761555	0.217810	84.542692	0	6	
2023-01-08 01:00:00	93.305281	-0.628516	91.891155	1	6	
2023-01-08 02:00:00	111.272907	-0.400570	88.969432	2	6	
2023-01-08 03:00:00	98.356280	0.541431	89.800948	3	6	
2023-01-08 04:00:00	105.521044	3.054382	84.002988	4	6	

	month	quarter	day_of_year	is_weekend	energy_lag_24h	\

Split & Baseline Models

1.6 6. Time-Based Split & Baseline Models

We'll split our data into training and testing sets based on time. We'll also create simple baseline models to compare against.

[9]:
```
split_date = '2023-10-01'
train_df = df.loc[df.index < split_date]
test_df = df.loc[df.index >= split_date]

X_train = train_df.drop('energy_kwh', axis=1)
y_train = train_df['energy_kwh']
X_test = test_df.drop('energy_kwh', axis=1)
y_test = test_df['energy_kwh']

# Baseline 1: Persistence Model (predicts the last known value)
y_pred_persist = X_test['energy_lag_24h'] # Using 24h lag as persistence

# Baseline 2: Seasonal Naive Model (predicts the value from the same time last␣
 ↪week)
y_pred_seasonal = X_test['energy_lag_1w']

print(f"Persistence MAE: {mean_absolute_error(y_test, y_pred_persist):.2f}")
print(f"Seasonal Naive MAE: {mean_absolute_error(y_test, y_pred_seasonal):.2f}")
```

```
Persistence MAE: 30.58
Seasonal Naive MAE: 11.36
```

ML Models

1.7 7. Machine Learning Models

Now, let's train OLS/Ridge and a Random Forest model.

:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

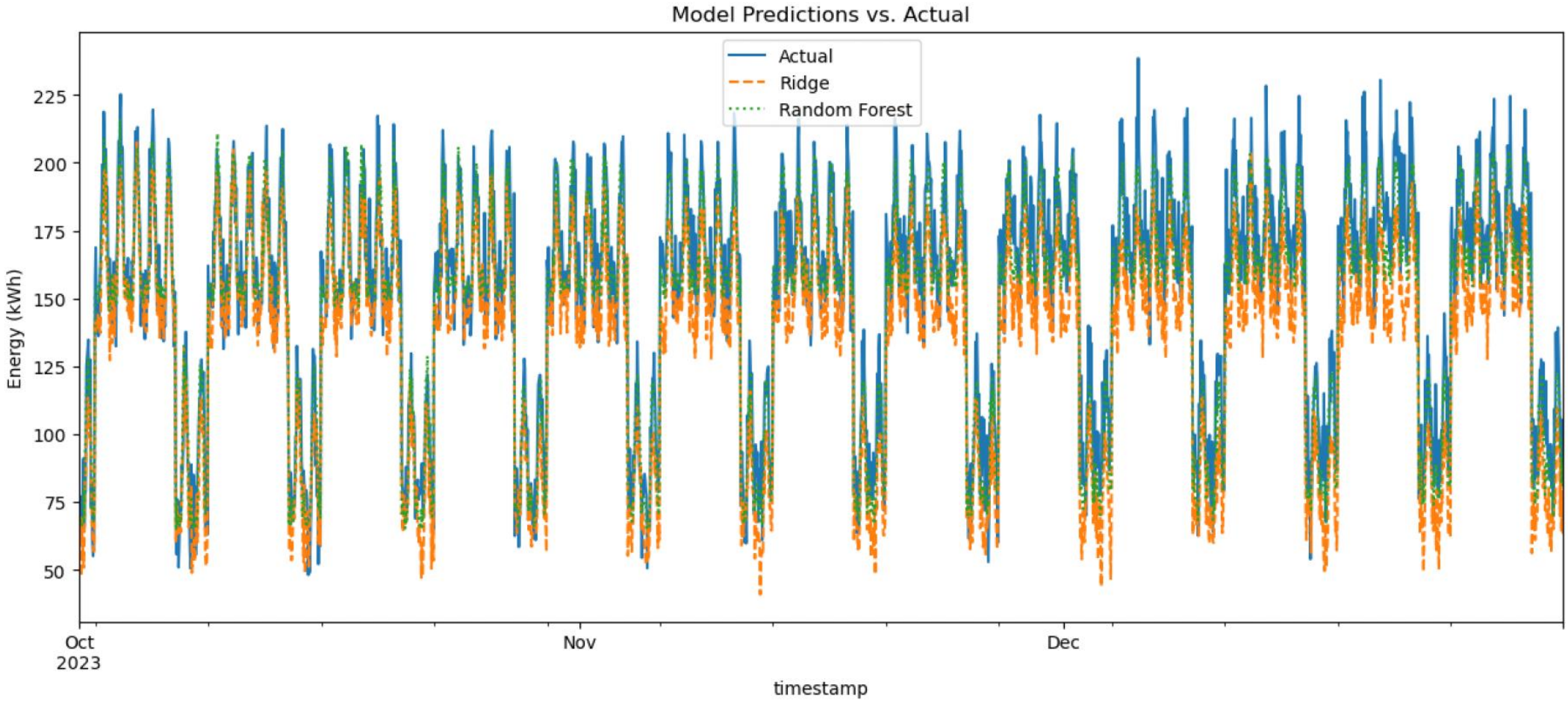
# OLS/Ridge Regression
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
y_pred_ridge = ridge.predict(X_test_scaled)

# Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1,␣
 ↪max_features='sqrt')
rf.fit(X_train, y_train) # RF doesn't require scaling
y_pred_rf = rf.predict(X_test)

print(f"Ridge MAE: {mean_absolute_error(y_test, y_pred_ridge):.2f}")
print(f"Random Forest MAE: {mean_absolute_error(y_test, y_pred_rf):.2f}")
```

```
Ridge MAE: 17.80
Random Forest MAE: 9.62
```

Error Evaluation & More ?



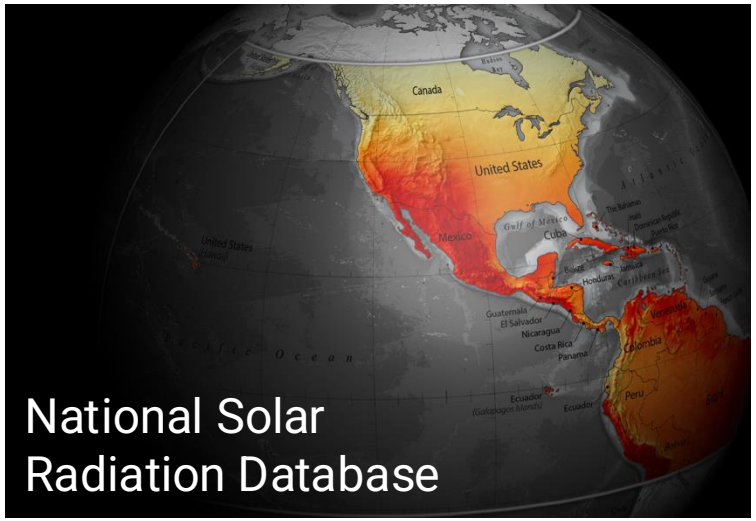
Part 3 - Mini Project

About Mini-Project

We will use Python to evaluate theoretical PV potential efficiency. I will provide the basic code and data before class, and introduce the code in class, including the meaning of each function and parameter, how to execute the code and obtain the evaluation results, and how to visualize the results.

Group and Report:

1. 3 students forms a group
2. Task:
    i. Modify the existing code to achieve the new requirement
    ii. Assess the PV potential using provided data and visualize the results;
    iii. Submit the mini project report
3. Submission: via canvas



National Solar Radiation Database

https://nsrdb.nrel.gov/

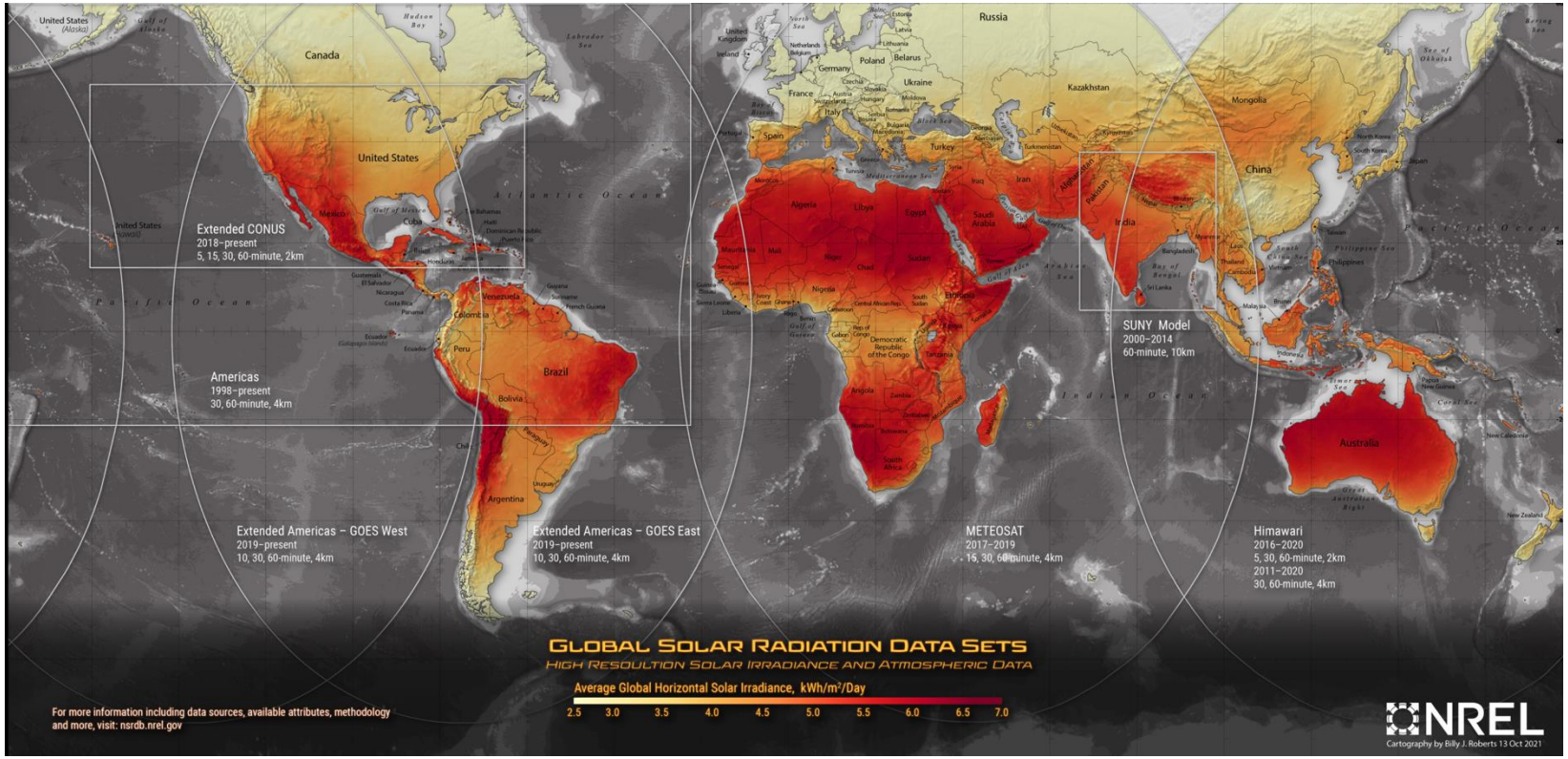
Before the lecture:

- Install Python on your computer
- Form the groups and ensure that at least one of the members brings the computer

About Mini-Project

1. About Mini-Project
2. Data and Collection Methods
3. Scripts and Results
4. Missions and Scores

Spatiotemporal Analysis of Photovoltaic Potential for Buildings

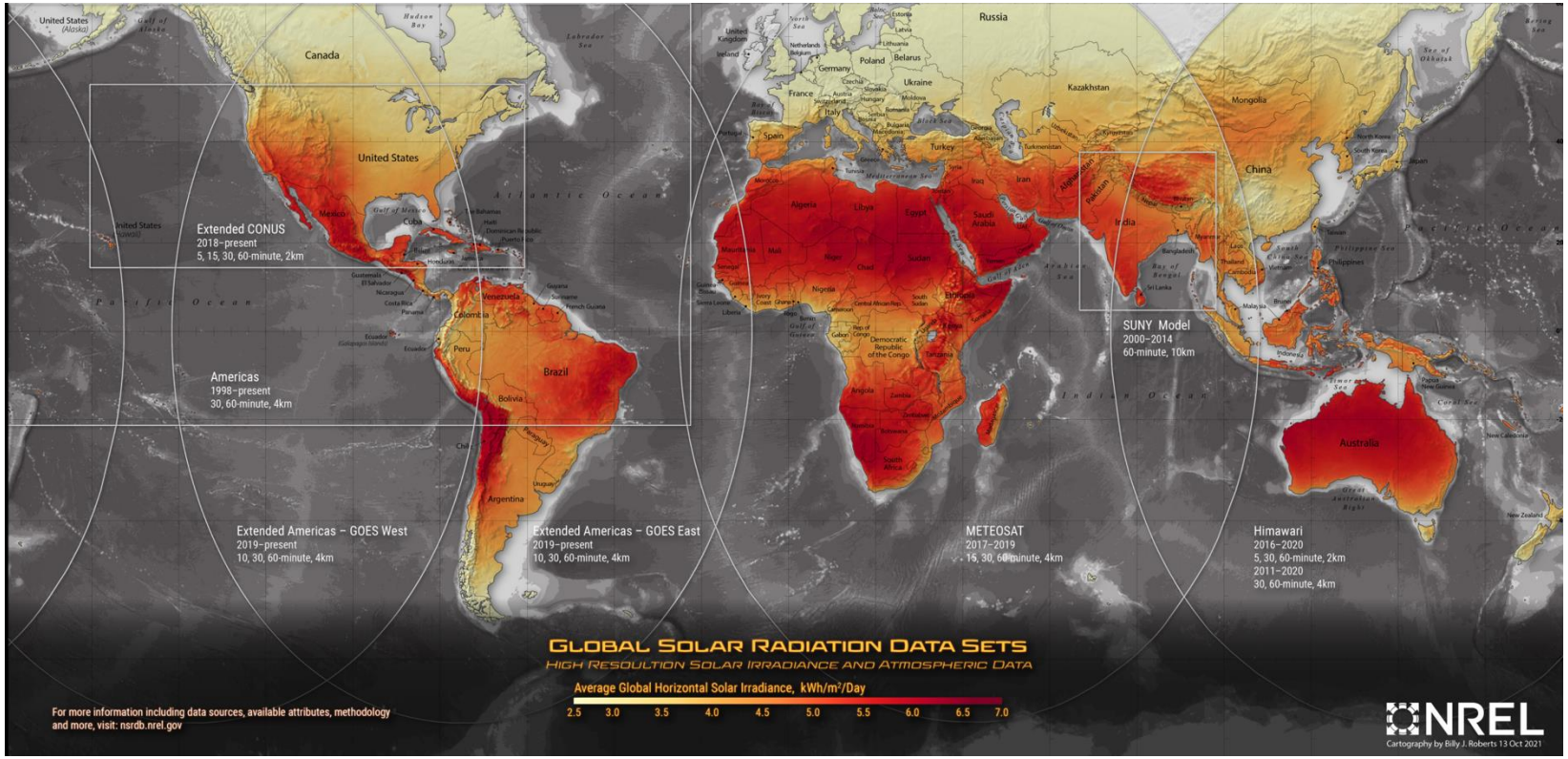


https://nsrdb.nrel.gov/about/what-is-the-nsrdb

About Mini-Project

1. About Mini-Project
2. Data and Collection Methods
3. Scripts and Results
4. Missions and Scores

Spatiotemporal Analysis of Photovoltaic Potential for Buildings



https://nsrdb.nrel.gov/about/what-is-the-nsrdb

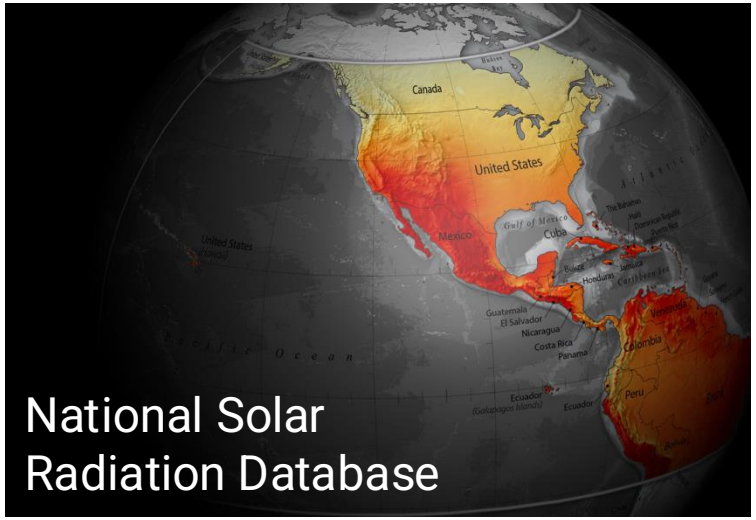
About Mini-Project

We will use Python to evaluate theoretical PV potential efficiency. I will provide the basic code and data before class, and introduce the code in class, including the meaning of each function and parameter, how to execute the code and obtain the evaluation results, and how to visualize the results.

Group and Report:

1. 3 students forms a group
2. Task:
   i. Modify the existing code to achieve the new requirement
   ii. Assess the PV potential using provided data and visualize the results;
   iii. Submit the mini project report
3. Submission:

via canvas before 26/10/26 (W9)



https://nsrdb.nrel.gov/

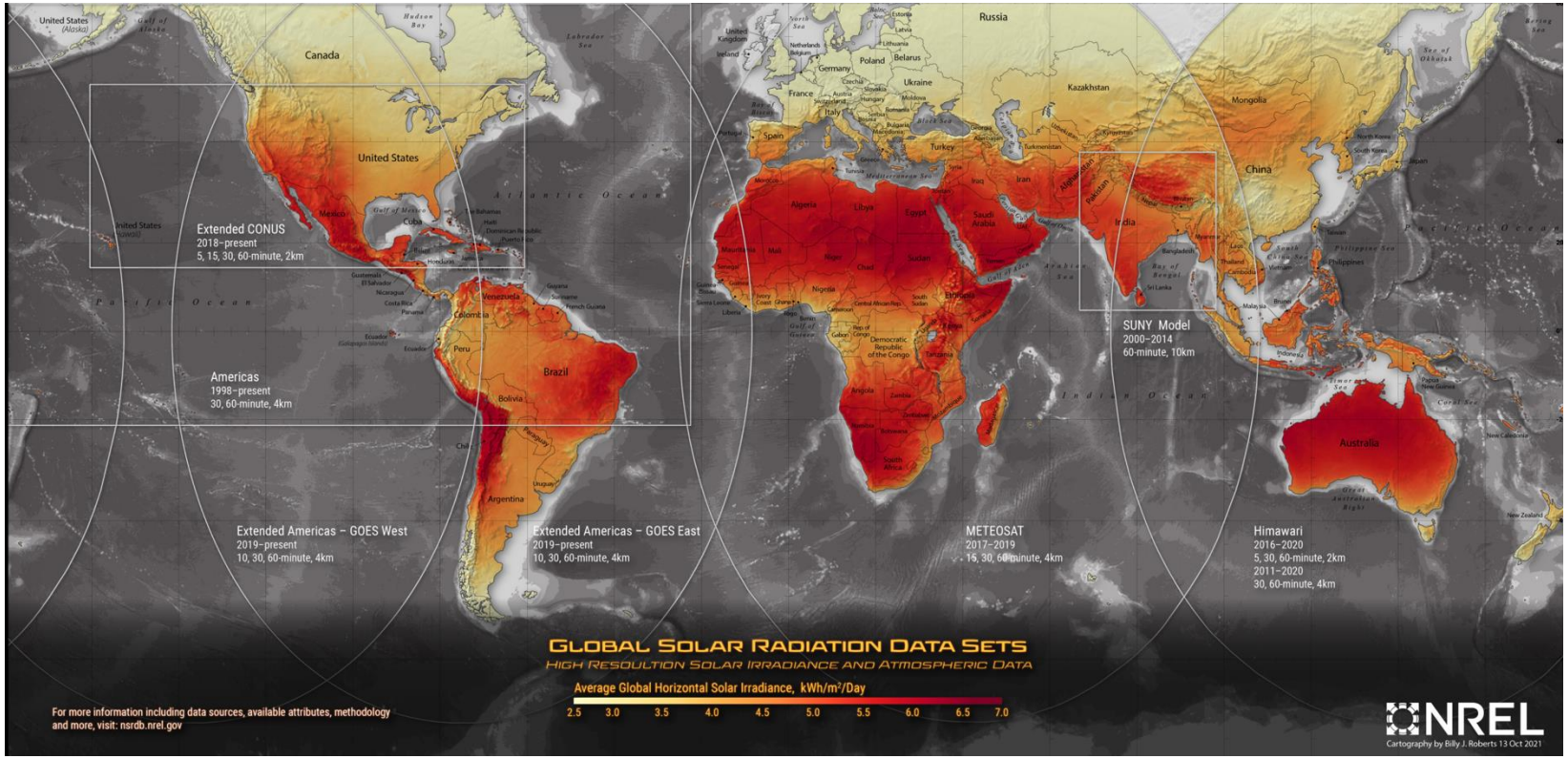
Before the lecture:

- Install Python on your computer
- Form the groups and ensure that at least one of the members brings the computer

Outline

1. About Mini-Project
2. Data and Collection Methods
3. Scripts and Results
4. Missions and Scores

Spatiotemporal Analysis of Photovoltaic Potential for Buildings



https://nsrdb.nrel.gov/about/what-is-the-nsrdb

Theoretical Potential: PoA

The Plane of Array (PoA) irradiance refers to the solar irradiance received on the tilted surface of a photovoltaic (PV) module. The calculation involves converting the measured horizontal solar irradiance into the irradiance received on the inclined PV panel.

- Direct Normal Irradiance (DNI)

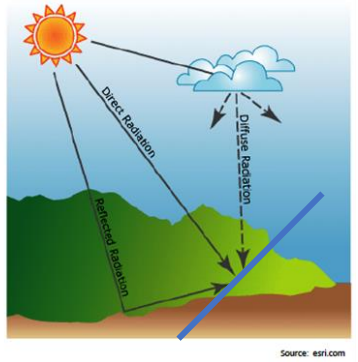


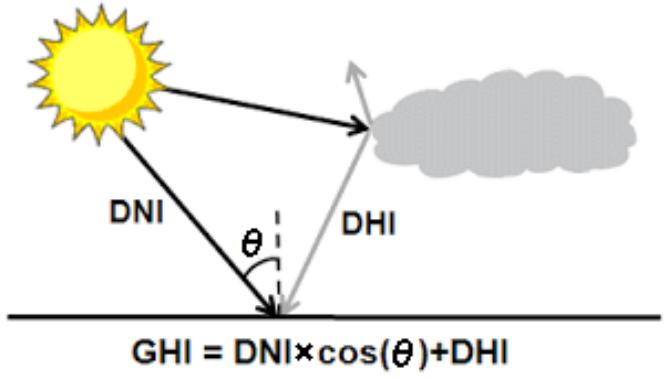
- Diffuse Horizontal Irradiance (DHI)



- Global Horizontal Irradiance (GHI)







Direct **Diffuse** **Reflected**

$$G_{PoA} = G_b \cos(\theta) + G_d \frac{1+\cos(\beta)}{2} + (G_b + G_d)\rho \frac{1-\cos(\beta)}{2}$$

Definitions of Each Parameter

Symbol	Meaning
G_{PoA}	Total irradiance on the tilted plane (W/m²)
G_b	Direct Normal Irradiance (DNI) (W/m²)
G_d	Diffuse Horizontal Irradiance (DHI) (W/m²)
θ	Incidence angle between solar rays and the module surface normal (degrees or radians)
β	Tilt angle of the module (degrees or radians)
ρ	Ground reflectance (Albedo), typically ranging from **0.2** (grass) to **0.6** (snow)

This formula accounts for three contributions:

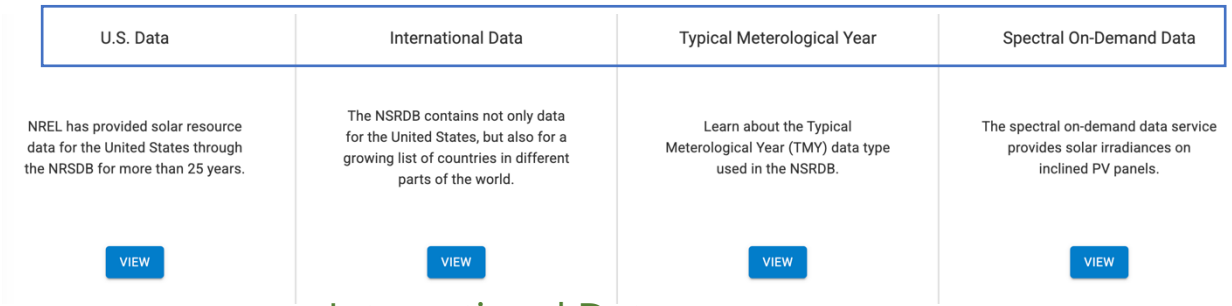
1. **Direct beam irradiance** projected onto the tilted surface ($G_b \cos(\theta)$).
2. **Diffuse irradiance** adjusted for the tilted plane ($G_d \frac{1+\cos(\beta)}{2}$).
3. **Ground-reflected irradiance** based on albedo ($(G_b + G_d)\rho \frac{1-\cos(\beta)}{2}$).

https://nsrdb.nrel.gov/data-viewer

NSRDB - National Solar Radiation Database

The National Solar Radiation Database (NSRDB) is a serially complete collection of hourly and half-hourly values of meteorological data and the three most common measurements of solar radiation: global horizontal (GHI), direct normal (DNI) and diffuse horizontal irradiance (DHI).

Data Sets



International Data

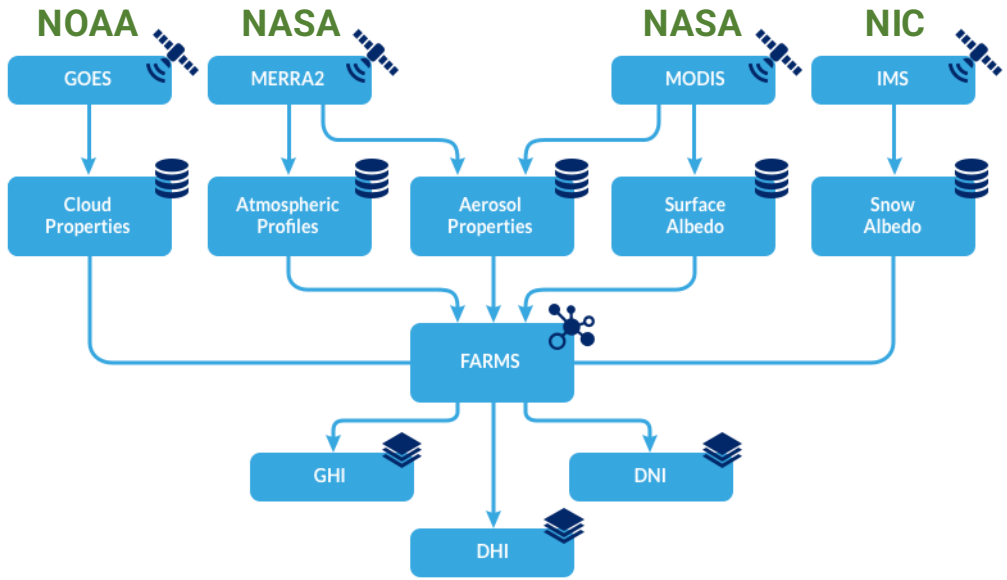
Region	Model Name	Satellite	Temporal Resolution	Spatial Resolution	Years Covered
Europe, Africa, & Asia	PSM V3	METEOSAT IODC	15, 30, 60-minute	4km	2017-2019
USA & Americas	PSM V3	GOES	30, 60-minute	4km	1998-2021
USA & Americas	PSM V3	GOES	10, 30, 60-minute	4km	2019-2021
USA (Continental) & Mexico	PSM V3	GOES	5, 30, 60-minute	2km	2019-2021
South Asia	SUNY	METEOSAT IODC	60-minute	10km	2000-2014
Asia, Australia & Pacific	PSM V3	Himawari	10, 30, 60-minute	2km	2016-2020
Asia, Australia & Pacific	PSM V3	Himawari	30, 60-minute	4km	2011-2015

https://nsrdb.nlr.gov/data-sets/international-data

Mini-Project

NSRDB - Data flow from satellite to solar radiation measurement

- Geostationary Operational Environmental Satellite (GOES) from National Oceanic and Atmospheric Administration (NOAA)
- Interactive Multisensor Snow and Ice Mapping System (IMS) from the National Ice Center (NIC)
- Moderate Resolution Imaging Spectroradiometer (MODIS) and Modern Era Retrospective analysis for Research and Applications, version 2 (MERRA-2) from the National Aeronautics and Space Administration (NASA)



Physical Solar Model (PSM), involves a two-step physical modeling process:

1. Deriving and resampling cloud and aerosol properties, and then utilizing the Fast All-sky Radiation Model for Solar applications (FARMS) for radiative transfer modeling.
2. By employing FARMS with Narrowband Irradiances for Tilted surfaces (FARMS-NIT), the NSRDB offers users spectral-on-demand irradiances tailored to specific time, location, and PV orientation selections.

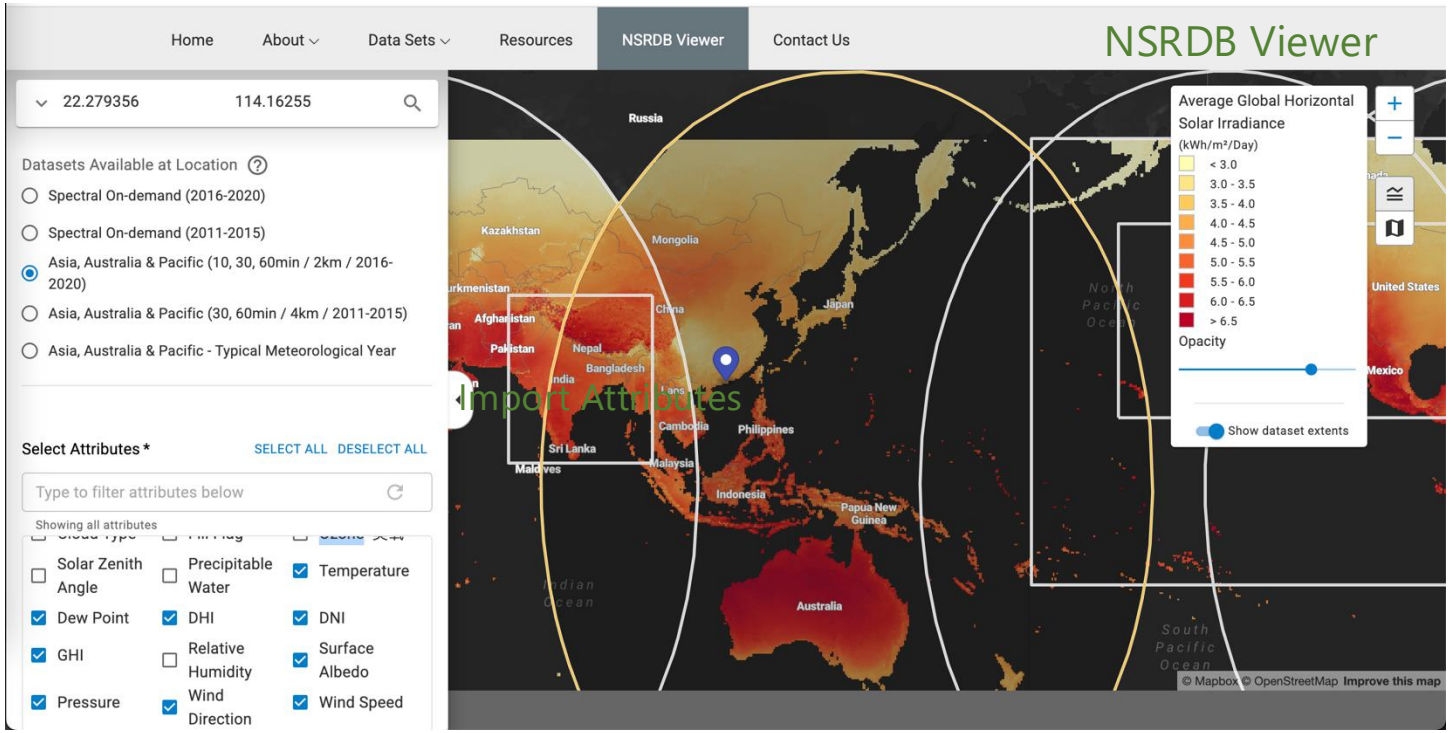
https://nsrdb.nrel.gov/about/what-is-the-nsrdb

Mini-Project

NSRDB - How to Access the Data

The NSRDB dataset can be accessed in a variety of ways:

- Directly from the NSRDB Viewer
- Programmatically using an application programming interface (API) - please refer to the NREL Developer Network for instructions
- AWS cloud service - please refer to the NSRDB GitHub Repo for instructions
- NREL's System Advisor Model



https://nsrdb.nrel.gov/data-viewer

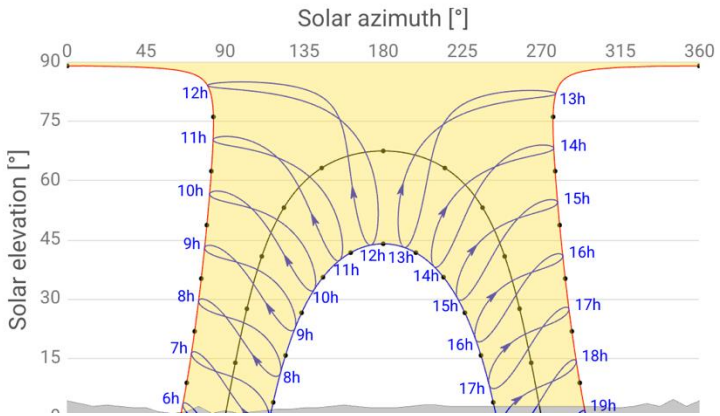
Mini-Project

Sample City - Shenzhen

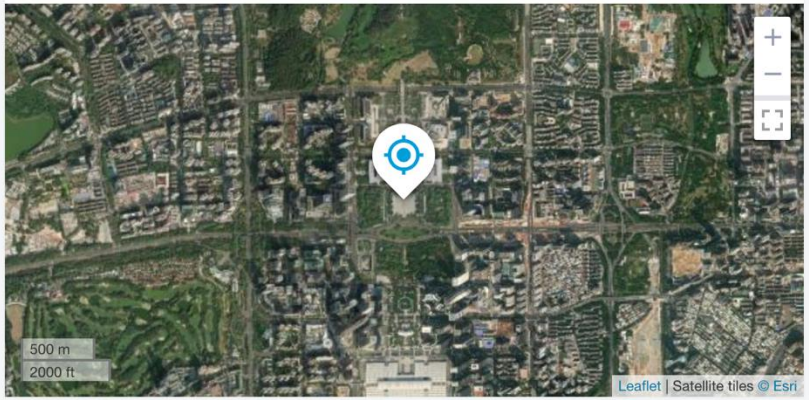
Map data Per year

Specific photovoltaic power output	PVOUT specific	1235.6	kWh/kWp
Direct normal irradiation	DNI	886.8	kWh/m²
Global horizontal irradiation	GHI	1470.2	kWh/m²
Diffuse horizontal irradiation	DIF	849.1	kWh/m²
Global tilted irradiation at optimum angle	GTI opta	1533.0	kWh/m²
Optimum tilt of PV modules	OPTA	19 / 180	°
Air temperature	TEMP	22.9	°C
Terrain elevation	ELE	N/A	

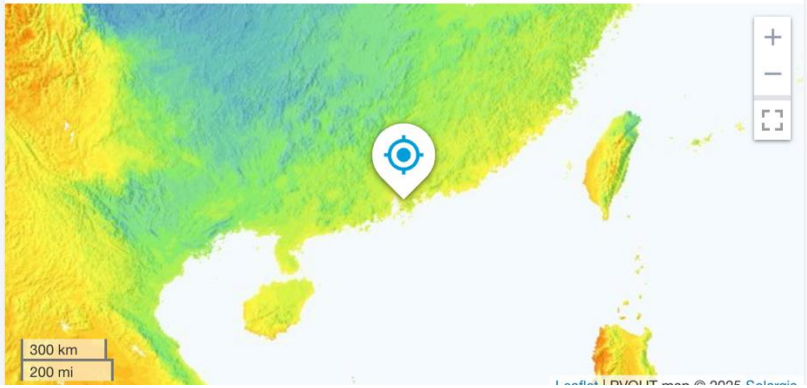
Horizon and sunpath



Map Swich to map



PVOUT map



https://globalsolaratlas.info/detail?c=22.544574,114.054543,11&s=22.544574,114.054543&m=site

Mini-Project

Shenzhen - Data Details

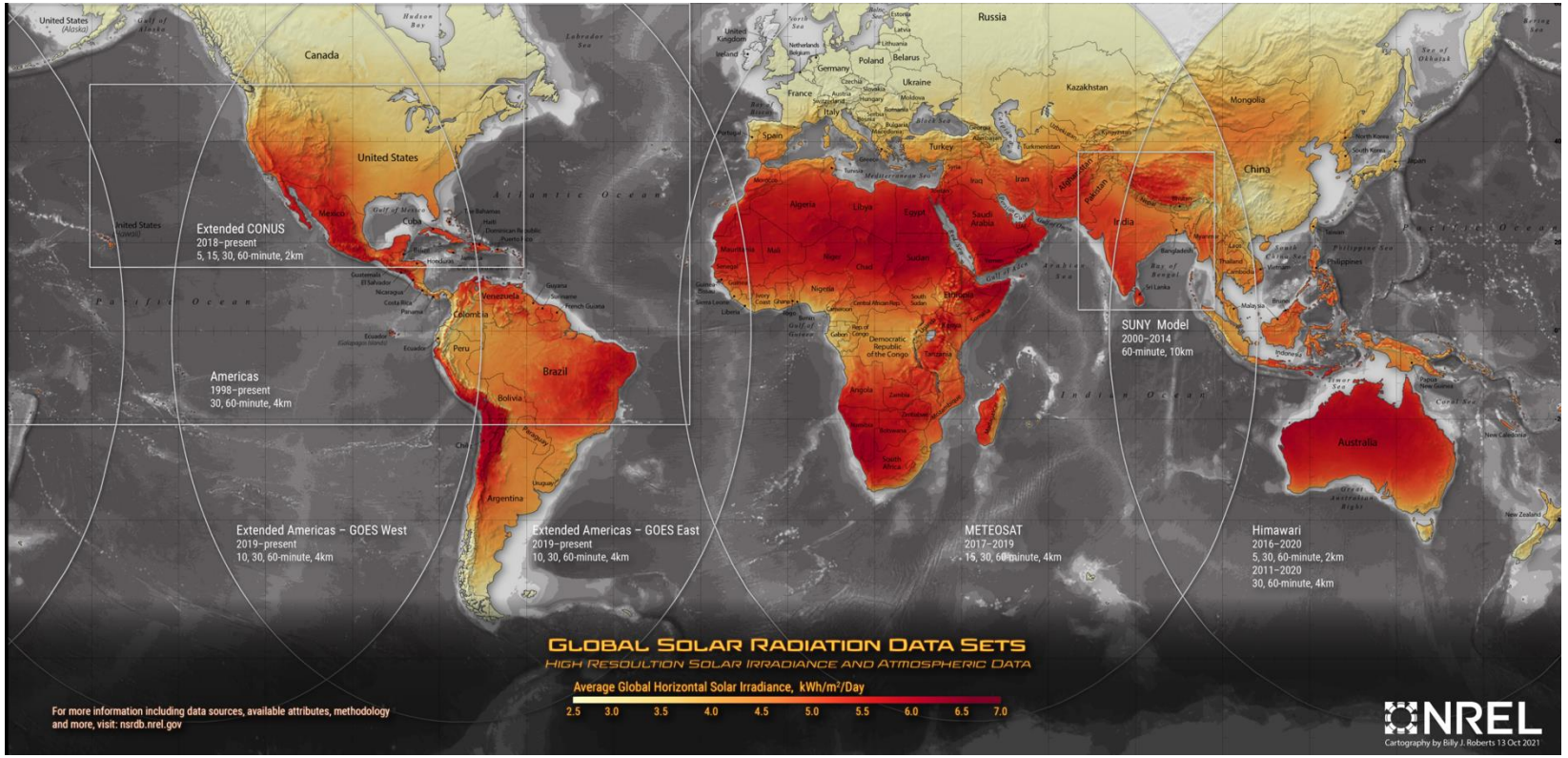


https://kepler.gl/demo

Outline

1. About Mini-Project
2. Data and Collection Methods
3. Scripts and Results
4. Missions and Scores

Spatiotemporal Analysis of Photovoltaic Potential for Buildings



https://nsrdb.nrel.gov/about/what-is-the-nsrdb

Outline

Conda Environment for Mini-Project



AI-Driven Programming IDE

Environment:
/opt/anaconda3/envs/miniproject/bin/python

Method 1

```
$ conda env create –f mini_project.yml
$ conda env list

# if ResolvePackageNotFound, try
$  conda env export --no-builds >
environment.yml
```

Method 2

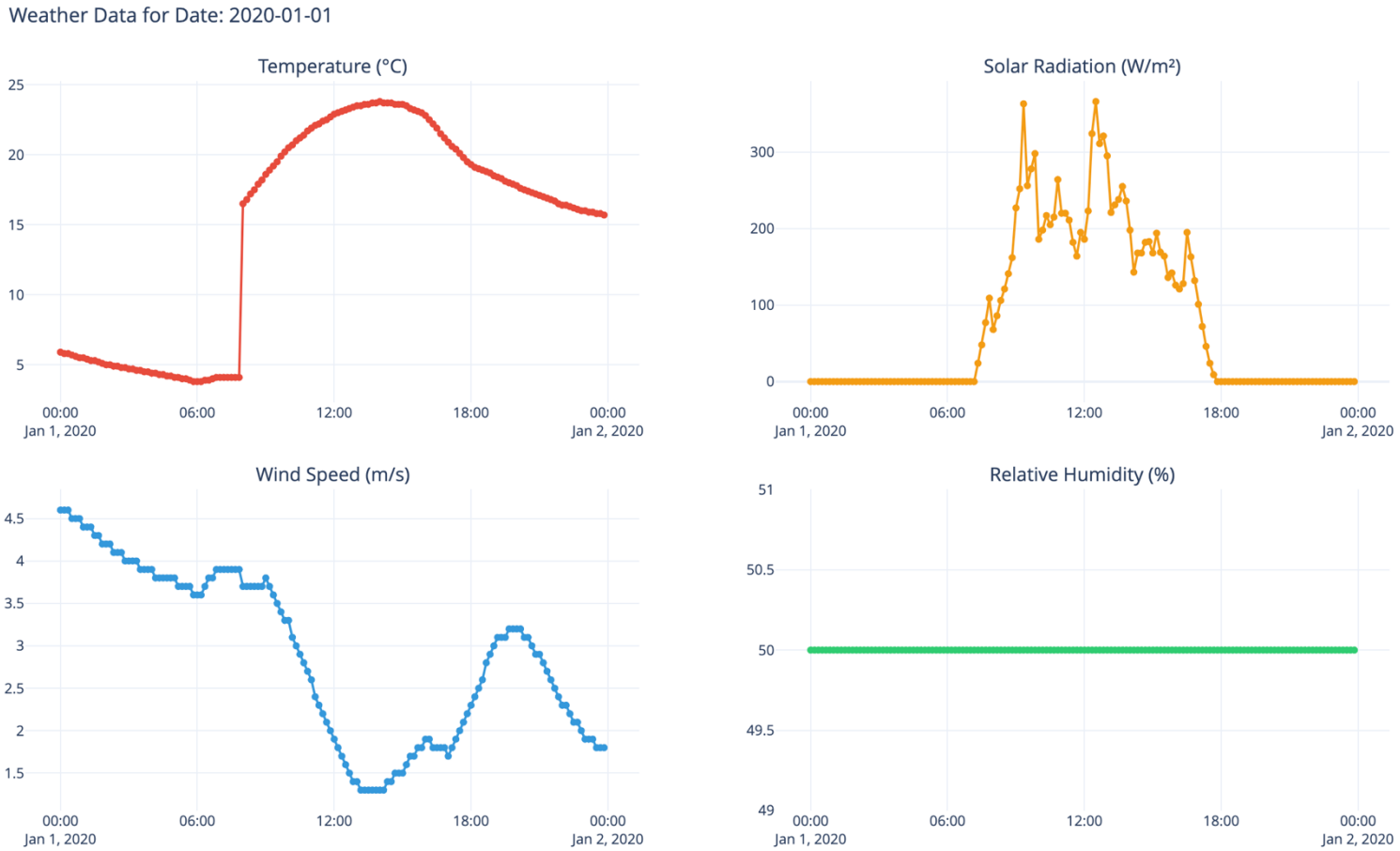
```
# conda create -n miniproject python=3.10
# condo activate miniproject

# pip install -r requirements.txt
```

https://www.cursor.com/cn/downloads

Shenzhen - Data Details

Data Visualization



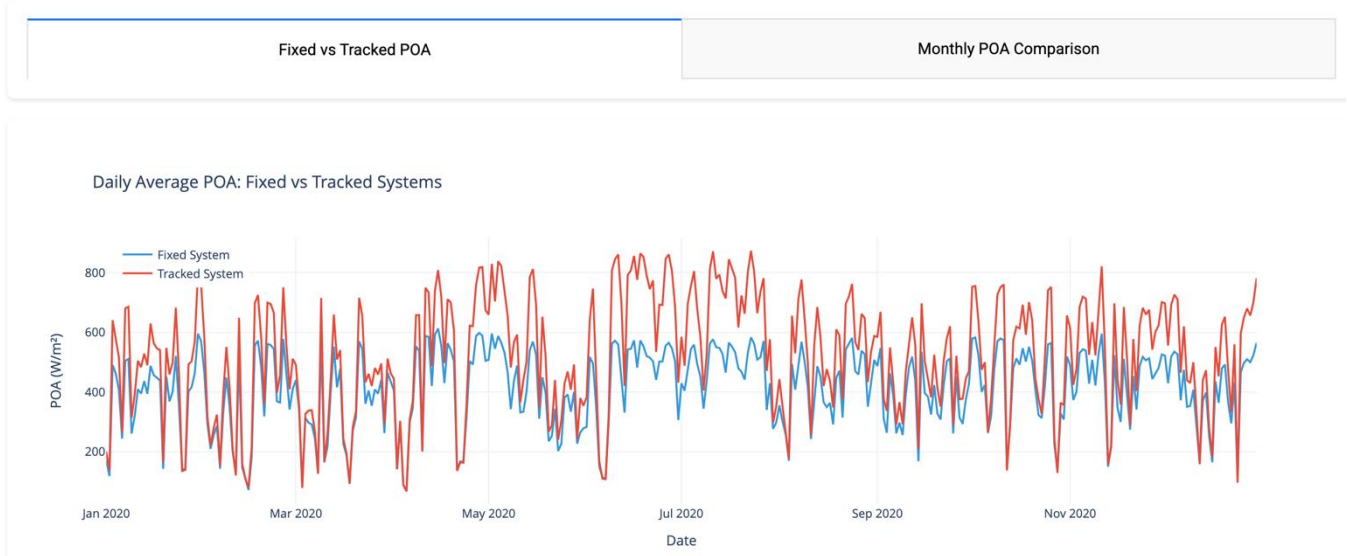
Shenzhen - Data Details

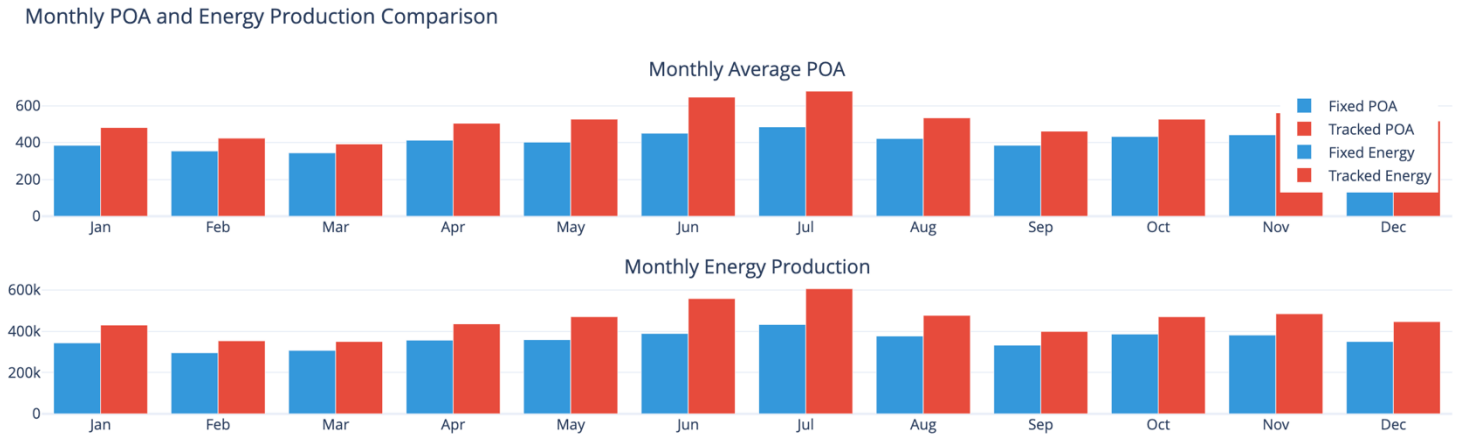
Data Visualization



Shenzhen - Data Details

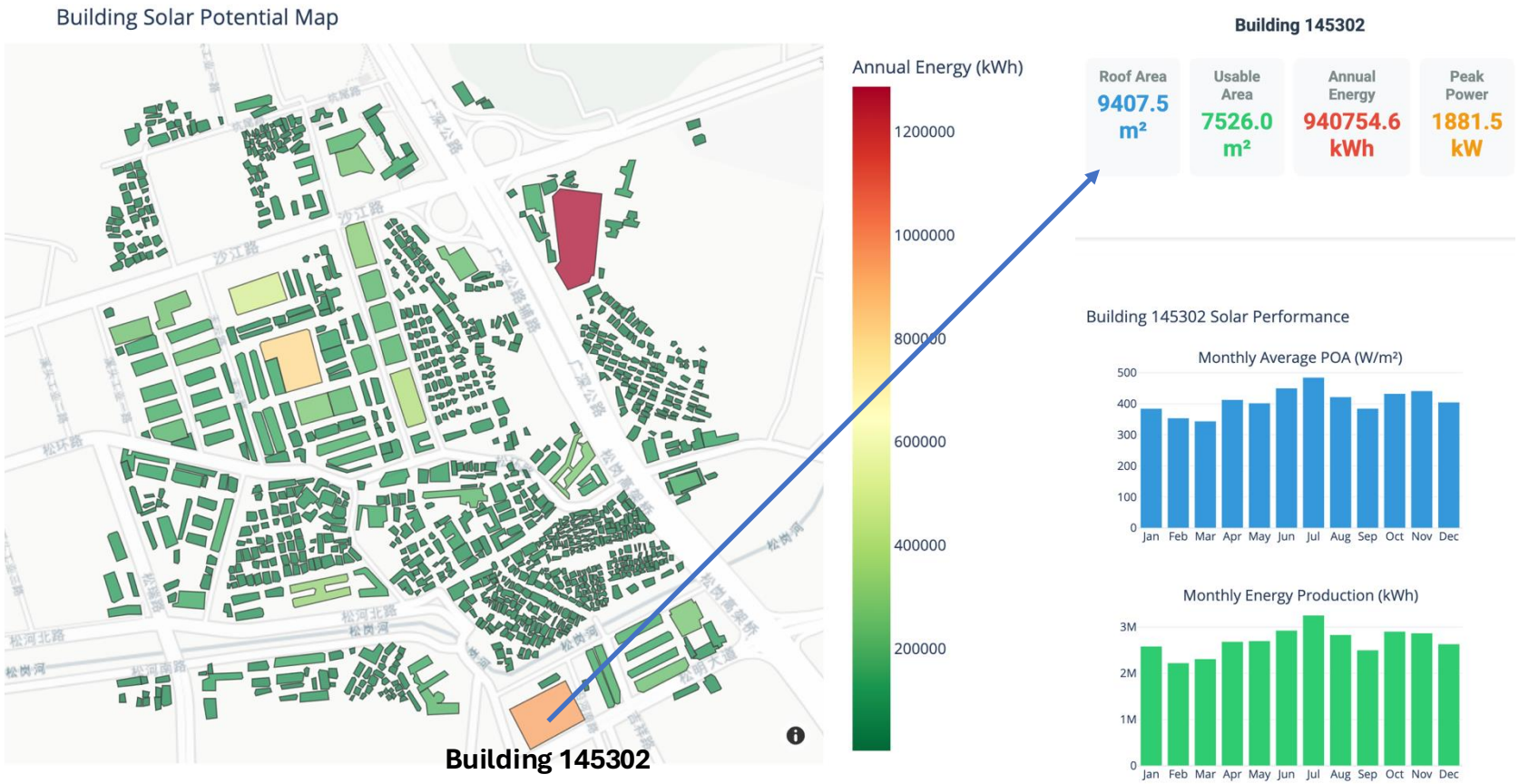
Solar POA Analysis





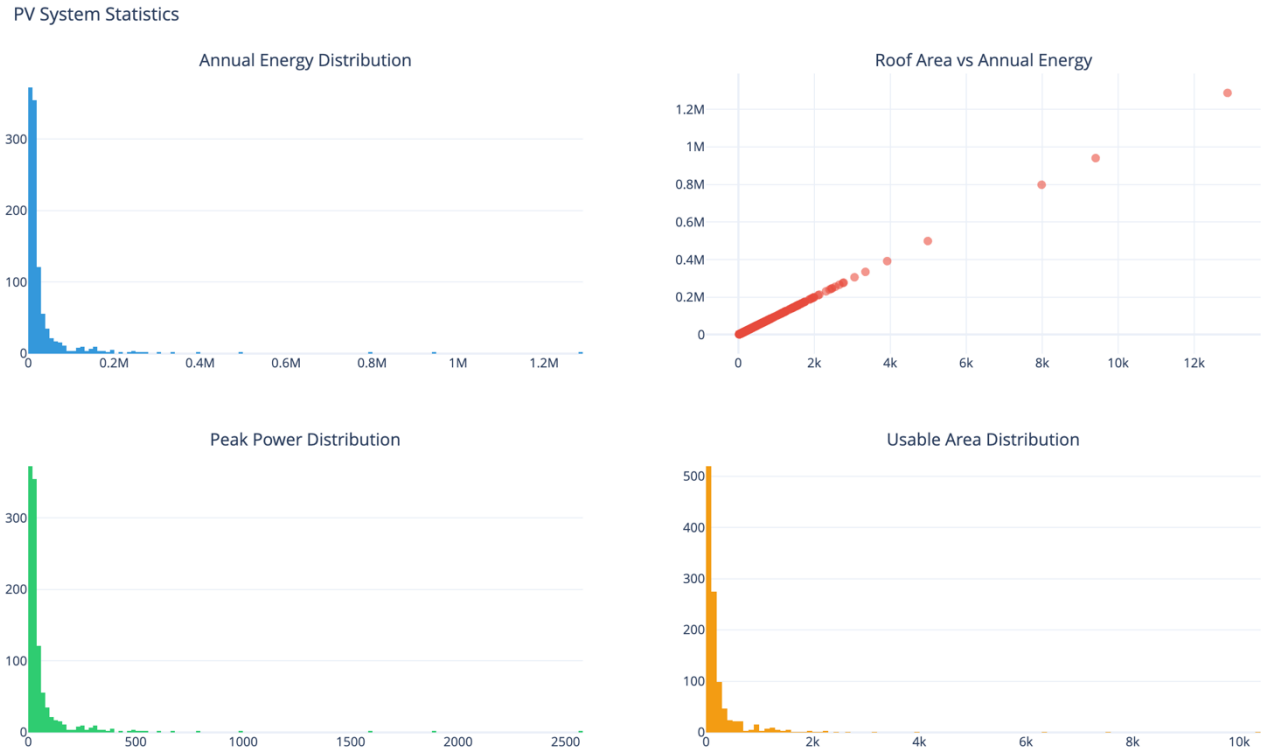
Shenzhen - Data Details

Building Analysis



Shenzhen - Data Details

PV System Statistics



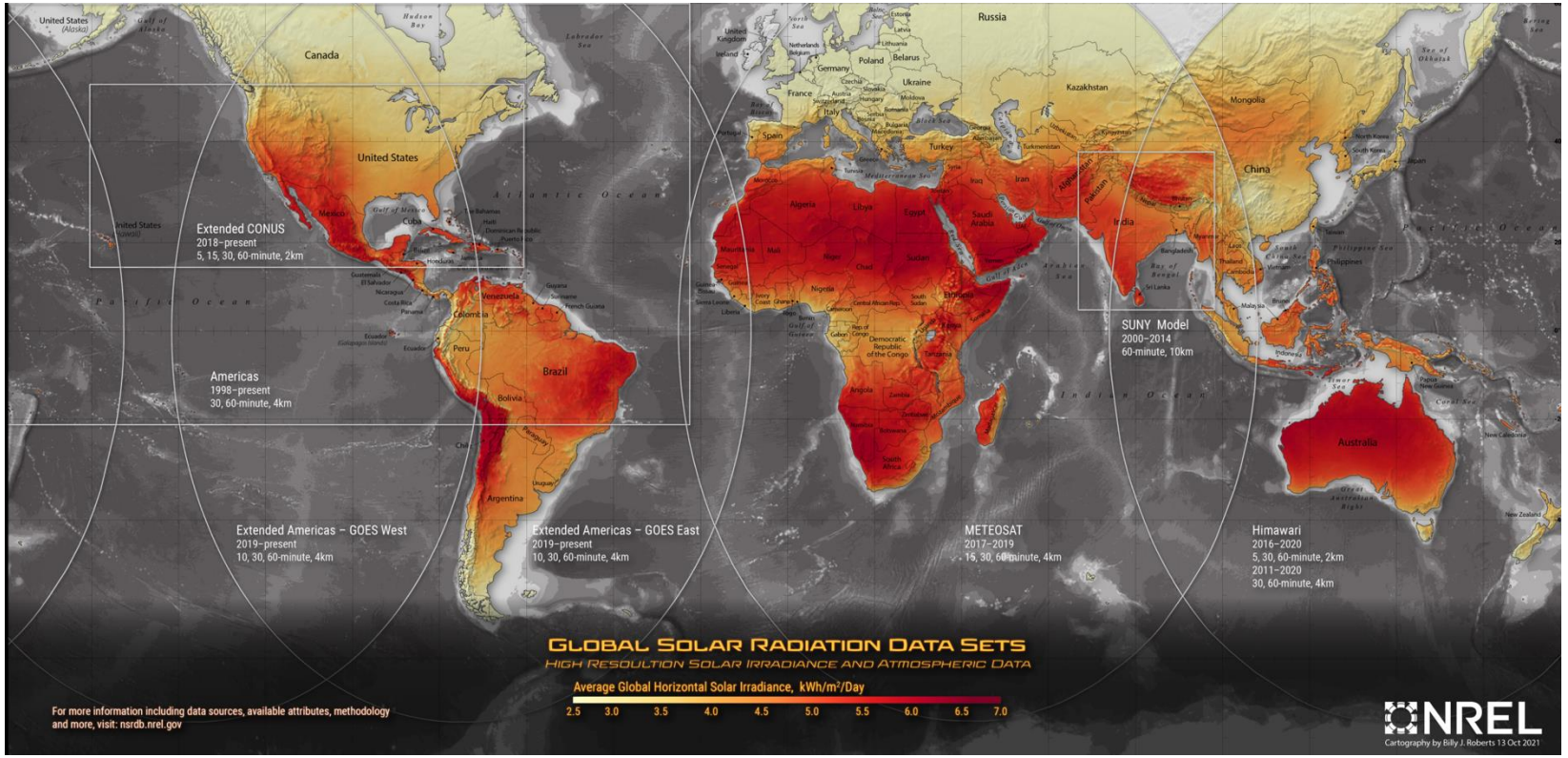
Summary Statistics

Total Annual Energy	Average Building Energy	Total Peak Power	Average Solar Radiation
33,698.6 MWh	31,202.4 kWh	67,397.2 kW	194.5 W/m²

Outline

1. About Mini-Project
2. Data and Collection Methods
3. Scripts and Results
4. Missions and Scores

Spatiotemporal Analysis of Photovoltaic Potential for Buildings



https://nsrdb.nrel.gov/about/what-is-the-nsrdb

Thanks
&
Any Question?

